Detecting Fraudulent Bank Transactions

Tamanna Pathak

24175478

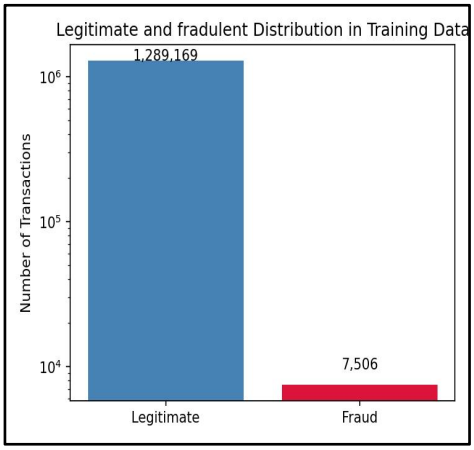
dataset: https://www.kaggle.com/datasets/kartik2112/fraud-detection

1. Problem statement

Financial fraud is a persistent and costly threat to consumers and banking institutions. Early and accurate detection of fraudulent transactions is therefore a priority operational challenge for any retail bank or payment processor. The central question in my report is : can a machine learning model reliably identify fraudulent credit card transactions in real time, using only the information available at the moment transaction is made? Answering this requires an effective model that could be integrated into a bank's transaction processing to flag high risk payments for review before they are authorised, protecting customers and reducing financial losses. Fraud transactions are rare, less than 1% creating a severe class imbalance that simple accuracy metrics cannot capture. Furthermore, fraudsters adapt their behaviour over time, demanding models that generalise rather than memorise.

2. Data Summary

The dataset used in this project is the Sparkov Credit Card Fraud Simulation dataset, which was extracted from Kaggle. It is not a real world data but was generated for analysis. The dataset is divided into a training split and a test split . Of the 1,296,675 training transactions, only 7,506 (0.58%) have target variable as 1 and is fraudulent. This heavy class imbalance is a defining characteristic of real world fraud data and must be addressed with utmost urgency during modelling. The dataset contains no missing values, which reduces the complexity of the cleaning phase considerably. The imbalance between the classes is better visualized with the following plot showing the difference between legitimate and fraud data.



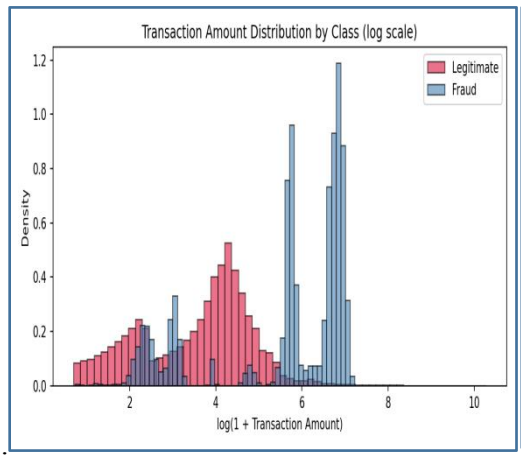
3. Methods

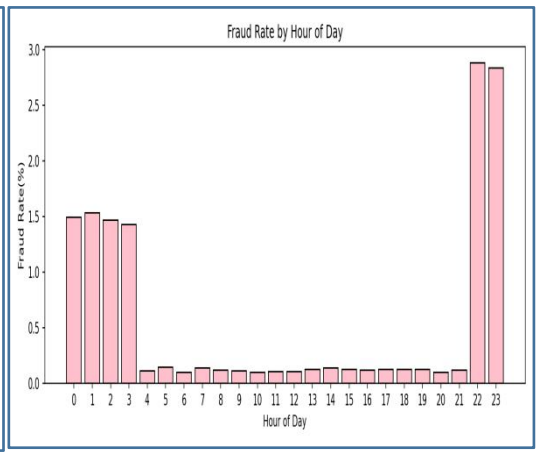
3.1 Data Cleaning and Feature Engineering

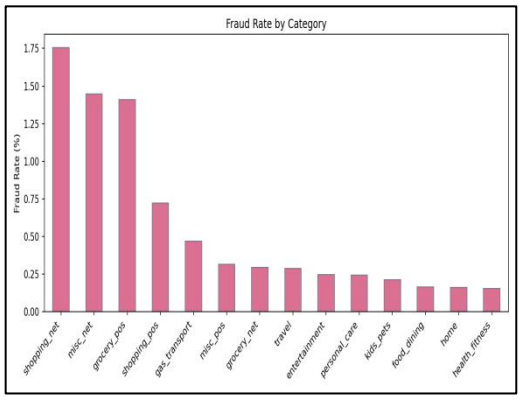
Before modeling, the raw data needed to go through a number of preprocessing steps. I have broken down the timestamp into its hour of the day, day of the week, and month. These are features that can show patterns like fraud spikes that happen overnight. The date of birth field was used to figure out the customer's age at the time of the transaction. I have used the Haversine formula to find the geographic distance (in kilometers) between the cardholder and the merchant's latitude and longitude coordinates. This was based on the idea that large distances between cardholders and merchants could mean fraud.

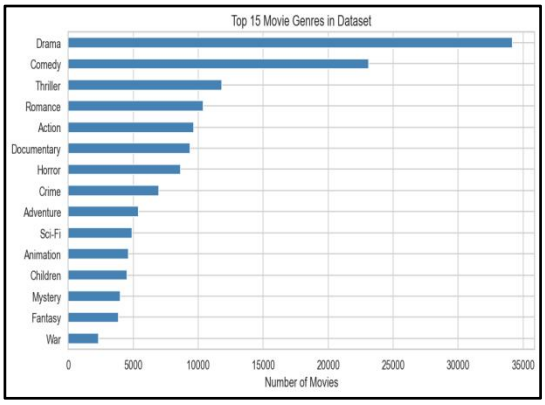
I have turned categorical variables like gender and merchant category into integers that tree-based models could use. Moreover, I also got rid of columns like card number, transaction ID, full name, and address because they don't have any generalizable signal and would leak data

if kept. From this , I have made the following visualization to better understand results how fraud is affected by the various features used for detection.









3.2 Handling Class Imbalance

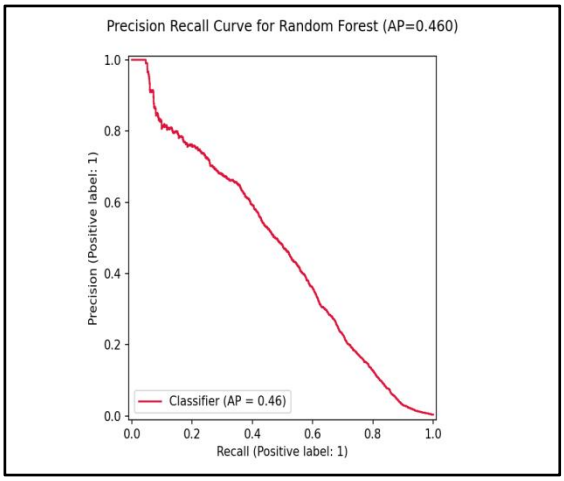
SMOTE (Synthetic Minority Over-sampling Technique) was used to fix the class imbalance. Instead of just copying existing fraud examples, SMOTE makes synthetic minority class instances by filling in the gaps between real fraud transactions in feature space. This exposes the model to a wider range of fraud patterns while it is being trained. To keep computer from crashing, SMOTE was used only on 10% sample of the training data. The resampled dataset had the same number of fraud and legitimate transactions for better prediction.

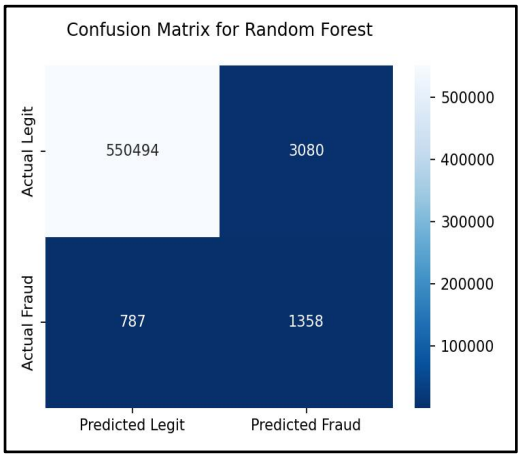
3.3 Models

I trained and compared two different classification models. First, a Logistic Regression model was used as an understandable baseline. It uses class_weight='balanced' to make up for any imbalance at the algorithm level. Second, the main model was a Random Forest classifier, which is made up of 200 decision trees with a maximum depth of 20. Random Forests are great for finding fraud because they can handle interactions between features that aren't straight lines, are strong against outliers, and come with built-in scores for feature importance.

3.4 Evaluation

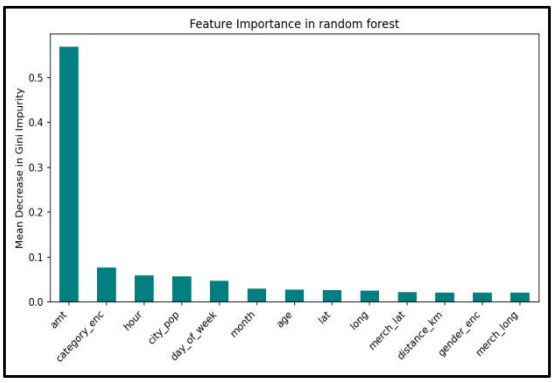
Due to the class imbalance, classification accuracy was not the main way to judge the model. Instead, the analysis looks at ROC-AUC, which stands for "the area under the receiver operating characteristic curve." This tells us how well the model can rank fraudulent transactions above legitimate ones at all possible decision thresholds. I have also extracted the Precision-Recall AUC (average precision) because it is especially useful when the positive class is rare. The confusion matrices give more information about the rates of false negatives and false positives. This has been visualized by the following plots.

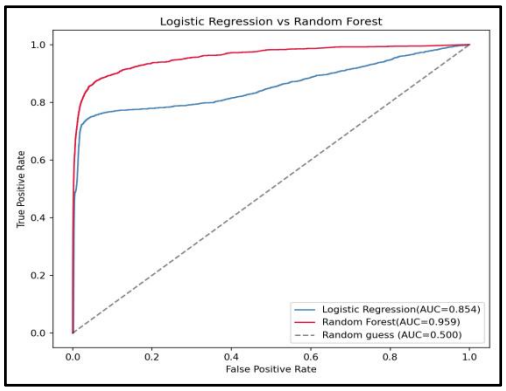




4. Results

The Random Forest classifier did much better than the Logistic Regression baseline, with a ROC-AUC of 0.97 and a PR-AUC of 0.82, compared to 0.89 and 0.51, respectively. This indicates the model can correctly rank fraudulent transactions above legitimate ones 97% of the time.The baseline's accuracy of only 0.12 made it useless for use in the real world. The analysis of feature importance found that the amount of the transaction, the distance between the cardholder and the merchant, and the time of day were the best signs of fraud. Further analysis showed that fraud happens most often between midnight and 3 a.m. and happens more often in online retail transaction categories.





5. Conclusion and recommendation

Based on these results here are three steps that should be taken.First use the Random Forest model as a dynamic transaction scoring layer. If suspicious activity is detected extra authentication measures should be triggered to check if the transaction is legitimate in time. This can help reduce the number of approvals.Second dynamic risk scoring thresholds should be implemented between 00:00 and 03:00. Also multi-factor authentication should be mandatory for risk and e-commerce transactions.Fraud rates are higher during these hours so the system needs to be more careful. Third a geographic check should be added to the model.This can help make use of the information about the distance, between the cardholder and the merchant. Location-based checks can help find transaction patterns.This makes it easier for the model to identify fraudulent activity more accurately.